

CSE Notes: Data Structures and Algorithms (DSA)

Exam-ready summary for library academic resource

1. What is DSA?

- Data Structure means a way of organizing and storing data so that operations can be performed efficiently.
- Algorithm means a finite step-by-step procedure to solve a problem.
- DSA helps to write programs that are faster, memory-efficient, and scalable.
- Common operations: insert, delete, search, update, traverse, sort, merge.

2. Time and Space Complexity

- Time complexity measures how running time grows with input size n .
- Space complexity measures how memory usage grows with input size n .
- Big-O notation gives the upper bound or worst-case growth rate.

Complexity	Name	Example
$O(1)$	Constant	Accessing <code>arr[i]</code>
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, heap sort
$O(n^2)$	Quadratic	Bubble sort, selection sort
$O(2^n)$	Exponential	Some recursive brute force
$O(n!)$	Factorial	Travelling salesman brute force

3. Arrays

- Array stores elements in contiguous memory locations.
- Index-based access is very fast: $O(1)$.
- Insertion/deletion in the middle is slow because shifting is needed: $O(n)$.
- Useful when size is known and frequent random access is needed.

Operation	Complexity
Access	$O(1)$
Search unsorted	$O(n)$
Search sorted with binary search	$O(\log n)$
Insert at end	$O(1)$ amortized
Insert/delete at middle	$O(n)$

4. Linked List

- Linked list stores data in nodes; each node contains data and pointer/link to next node.
- Types: singly linked list, doubly linked list, circular linked list.
- Insertion and deletion are efficient if position/node reference is known.
- Random access is not possible; traversal is required.

Type	Structure	Use
Singly	Data + next	Simple dynamic list
Doubly	prev + data + next	Forward and backward traversal
Circular	Last node links to first	Round-robin scheduling

5. Stack

- Stack follows LIFO: Last In, First Out.
- Main operations: push, pop, peek/top.
- Applications: function call stack, undo operation, expression conversion, parenthesis matching, DFS.

Operation	Meaning	Complexity
push	Insert item at top	O(1)
pop	Remove item from top	O(1)
peek	Read top item	O(1)

Parenthesis Matching Idea

- Scan expression from left to right.
- Push opening brackets into stack.
- For closing bracket, check whether stack top matches.
- At the end, stack must be empty for a balanced expression.

6. Queue

- Queue follows FIFO: First In, First Out.
- Main operations: enqueue, dequeue, front/peek.
- Applications: CPU scheduling, printer queue, BFS, buffer management.

Queue Type	Meaning
Simple Queue	Insertion at rear, deletion at front
Circular Queue	Last position connects to first position to reuse space
Priority Queue	Element with highest priority is served first
Deque	Insertion and deletion possible at both ends

7. Recursion

- Recursion is a technique where a function calls itself.
- Every recursive function must have a base case to stop recursion.
- Recursive case reduces the problem into a smaller subproblem.
- Recursion is useful for tree traversal, divide and conquer, backtracking, and factorial/Fibonacci problems.

Example pseudocode:

```
factorial(n):  
    if n == 0 or n == 1: return 1  
    return n * factorial(n - 1)
```

8. Searching Algorithms

Algorithm	Requirement	Time Complexity	Idea
Linear Search	No sorting required	O(n)	Check each element one by one
Binary Search	Array must be sorted	O(log n)	Repeatedly divide search range into half

Binary Search Steps

- Set low = 0 and high = n - 1.
- Find mid = (low + high) // 2.
- If arr[mid] is target, answer found.
- If target is smaller, search left half.

- If target is larger, search right half.

9. Sorting Algorithms

Sorting Algorithm	Best	Average	Worst	Stable?	Main Idea
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Repeatedly swap adjacent wrong elements
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Select minimum and place it at correct position
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Insert each item into sorted part
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	Divide array, sort halves, merge
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No	Partition around pivot
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No	Use heap structure

10. Hashing

- Hashing maps a key to an index using a hash function.
- Average-case search, insert, and delete can be $O(1)$.
- Collision occurs when two keys map to the same index.
- Collision handling methods: chaining and open addressing.

Term	Meaning
Hash Function	Converts key into array index
Hash Table	Array-like structure that stores key-value data
Collision	Two or more keys get same hash index
Load Factor	Number of stored elements / table size

11. Trees

- Tree is a non-linear hierarchical data structure.
- Important terms: root, parent, child, leaf, edge, height, depth, subtree.
- Binary tree: each node has at most two children.
- Binary Search Tree (BST): left child values are smaller and right child values are larger than root.

Traversal	Order
Preorder	Root - Left - Right
Inorder	Left - Root - Right
Postorder	Left - Right - Root
Level Order	Visit nodes level by level using queue

- In a BST, inorder traversal gives sorted order.
- Balanced trees keep operations near $O(\log n)$. Unbalanced BST can become $O(n)$.

12. Heap and Priority Queue

- Heap is a complete binary tree generally stored in an array.
- Max-heap: parent is greater than or equal to children.
- Min-heap: parent is smaller than or equal to children.
- Priority queue is commonly implemented using heap.

Operation	Heap Complexity
Insert	$O(\log n)$
Extract min/max	$O(\log n)$

Peek min/max	$O(1)$
Build heap	$O(n)$

13. Graphs

- Graph consists of vertices/nodes and edges.
- Types: directed, undirected, weighted, unweighted, cyclic, acyclic, connected, disconnected.
- Representation: adjacency matrix and adjacency list.
- BFS uses queue and DFS uses stack/recursion.

Representation	Space	Good For
Adjacency Matrix	$O(V^2)$	Dense graph and fast edge checking
Adjacency List	$O(V + E)$	Sparse graph and efficient traversal

Algorithm	Data Structure	Use	Complexity
BFS	Queue	Shortest path in unweighted graph	$O(V + E)$
DFS	Stack/Recursion	Traversal, cycle detection, topological sort	$O(V + E)$
Dijkstra	Priority Queue	Shortest path with non-negative weights	$O((V+E) \log V)$
Kruskal	Disjoint Set	Minimum Spanning Tree	$O(E \log E)$
Prim	Priority Queue	Minimum Spanning Tree	$O(E \log V)$

14. Dynamic Programming Basics

- Dynamic Programming (DP) solves problems by storing results of overlapping subproblems.
- Two approaches: memoization (top-down) and tabulation (bottom-up).
- DP is useful when the problem has overlapping subproblems and optimal substructure.
- Common DP problems: Fibonacci, knapsack, longest common subsequence, coin change.

15. Quick Exam Revision Sheet

Topic	Must Remember
Array	Fast access $O(1)$, slow middle insertion/deletion $O(n)$
Linked List	Dynamic size, no random access
Stack	LIFO, push/pop $O(1)$
Queue	FIFO, enqueue/dequeue $O(1)$
Binary Search	Sorted array required, $O(\log n)$
Merge Sort	Stable, $O(n \log n)$, needs extra memory
Quick Sort	Average $O(n \log n)$, worst $O(n^2)$
Hash Table	Average $O(1)$, collision possible
BST	Inorder gives sorted order
Graph BFS	Uses queue, finds shortest path in unweighted graph
Graph DFS	Uses stack/recursion, useful for cycle detection

16. Common Viva/Exam Questions

Q: What is the difference between data structure and algorithm?

A: Data structure organizes data; algorithm is a step-by-step method to solve a problem.

Q: Why is binary search faster than linear search?

A: Binary search reduces the search space by half each step, so it takes $O(\log n)$ time.

Q: Why is stack called LIFO?

A: The last element inserted is the first element removed.

Q: What is the main difference between stack and queue?

A: Stack follows LIFO, queue follows FIFO.

Q: What is collision in hashing?

A: Collision occurs when two keys produce the same hash index.

Q: Why is inorder traversal important in BST?

A: It visits the BST elements in sorted order.

Q: Which data structure is used in BFS?

A: Queue.

Q: Which data structure is used in DFS?

A: Stack or recursion.